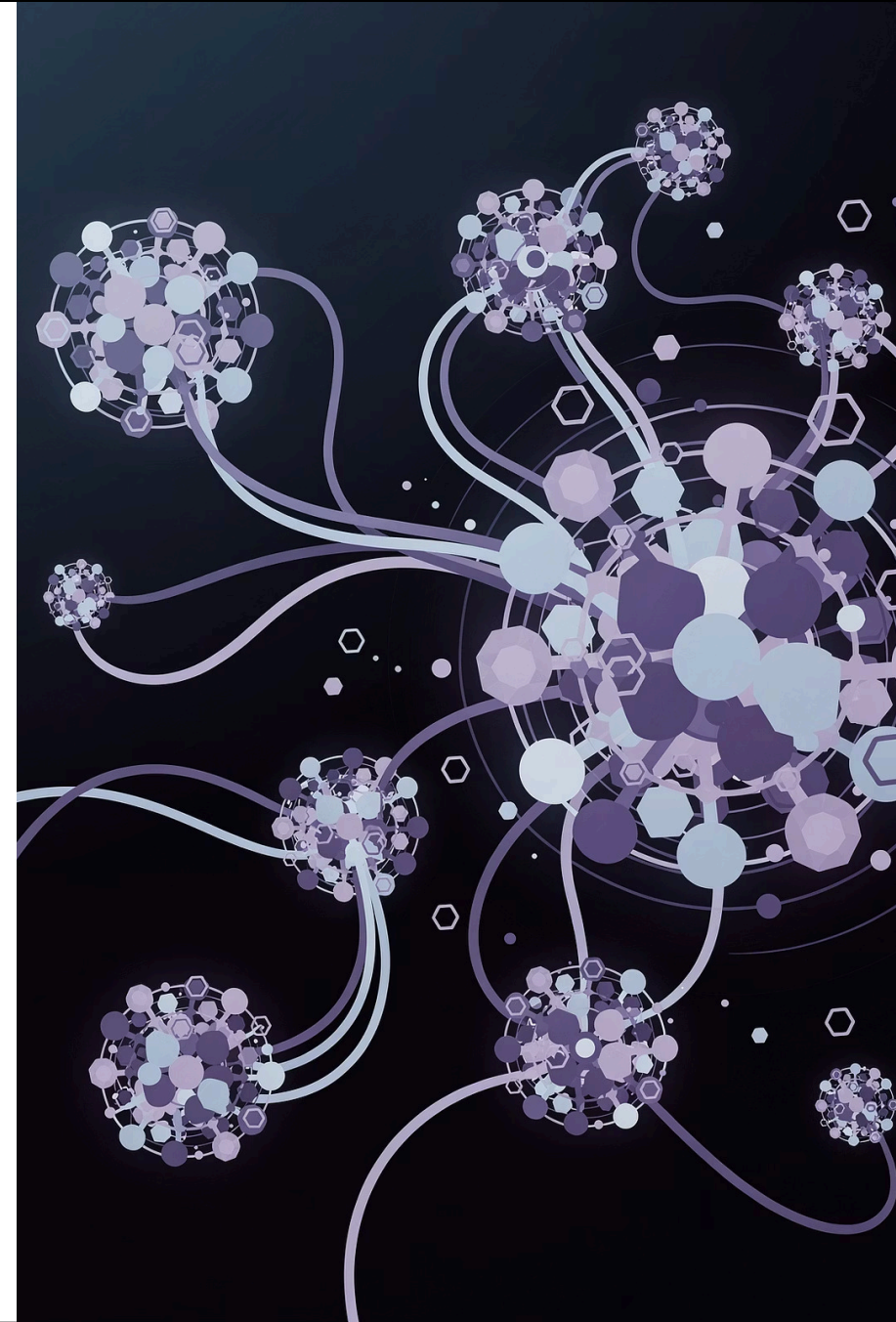


Module 02

Tokens, Embeddings & Vector Semantics

Understanding how machines transform language into mathematical representations they can process and compare

 GENAI, NLP & PROMPT ENGINEERING



Quick Recap: Module 01 Key Insights

Probability, Not Facts

GenAI operates on statistical predictions, not stored knowledge databases. It predicts the next most likely token based on patterns learned during training.

Mathematical Processing

Language models convert all text into numbers and vectors, performing complex mathematical operations to generate responses.

Hallucination Risk

Without proper grounding in verified data, models can generate plausible-sounding but factually incorrect information.

Responsible AI Use

Ethical considerations, bias awareness, and verification of outputs are essential practices when working with generative AI systems.

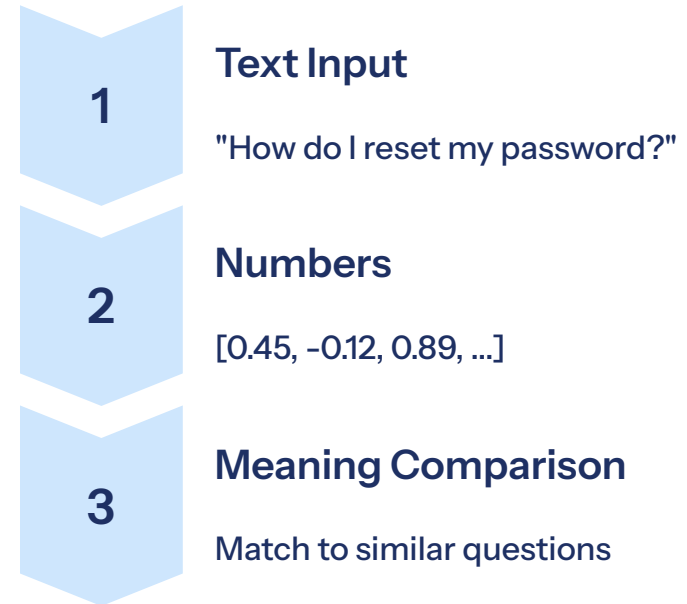
Module Overview: The Bridge Between Language and Math

The Core Challenge

Machines don't understand words the way humans do. They can't grasp meaning directly from text. To process language, AI systems must first convert every word, sentence, and document into numerical representations that can be mathematically manipulated and compared.

Learning Objectives

- Understand how text is broken into **tokens** for processing
- Learn how **embeddings** capture semantic meaning
- Explore **vector space** and similarity measurement
- Connect these concepts to chatbots and RAG systems



What is a Token?

Definition & Importance

A **token** is the smallest unit that a language model processes. Think of it as the atomic building block of AI text processing. Every input you provide—whether a single word, a sentence, or an entire document—is broken down into these fundamental units before any processing begins.

Tokens aren't always complete words. They can be:

- **Whole words:** "cat", "running", "computer"
- **Sub-word pieces:** "un" + "believe" + "able"
- **Punctuation:** "!", "!", "?"
- **Numbers:** "2024", "100"

Tokenization Example



Input sentence:

"Tokenization helps AI understand text."

Broken into tokens:

- "Token" + "ization"
- "helps"
- "AI"
- "understand"
- "text"
- "."

Different models use different tokenization strategies. GPT-4 might split words differently than Claude or Gemini, affecting how they interpret your prompts.

Why Tokenization Matters

Prompt Length & Cost

API providers charge based on token count, not character count. A 100-word prompt might consume 120-150 tokens depending on complexity. Understanding tokenization helps you optimize costs and stay within budget constraints.

Context Window Usage

Models have fixed token limits (e.g., 8,000 or 128,000 tokens). Your prompt, conversation history, and response all share this budget. Inefficient tokenization means you hit limits faster, losing valuable context.

Meaning Preservation

Poor token splits can break semantic meaning. If "unhappiness" becomes "un" + "hap" + "pin" + "ess", the model may struggle to understand the intended concept, leading to degraded performance.

Accuracy Impact

Efficient tokenization improves model accuracy. When tokens align with meaningful linguistic units, the model can better capture relationships between concepts and generate more relevant responses.

What are Embeddings?

From Text to Meaningful Numbers

Embeddings are numerical representations that capture the *meaning* of text. While tokens break text into pieces, embeddings transform those pieces into vectors—lists of numbers—that encode semantic information.

Here's the magic: words with similar meanings get similar vectors. "Cat" and "dog" will have vectors that are mathematically close to each other, while "cat" and "car" will be farther apart. This numerical proximity mirrors semantic similarity.

Why This Matters

Embeddings enable machines to:

- Compare words and sentences mathematically
- Search for relevant information based on meaning
- Retrieve documents that match user intent
- Power recommendation systems and semantic search



Simple Vector Example

"cat" → [0.8, 0.6, 0.1, -0.2]

"dog" → [0.75, 0.65, 0.15, -0.18]

"car" → [-0.3, 0.1, 0.9, 0.7]

Notice how cat and dog vectors are similar (both animals), while car is quite different (vehicle).



Sentence Embeddings: Capturing Complete Meaning

While word embeddings represent individual tokens, **sentence embeddings** compress entire sentences or paragraphs into single vectors. This aggregation combines the meaning of all constituent tokens, preserving context and semantic nuance.

How They Work

Sentence embeddings don't just average word vectors. They use sophisticated neural networks that understand word order, context, and relationships. The result is a fixed-length vector that represents the holistic meaning of the input text, enabling direct comparison between questions, documents, and other text spans.

Sentence	Embedding (simplified)	Use Case
"How do I reset my password?"	[0.65, -0.34, 0.89, 0.12, ...]	FAQ matching, support automation
"What's the process for password recovery?"	[0.63, -0.32, 0.91, 0.15, ...]	Duplicate detection, routing
"Tell me about your company history."	[-0.12, 0.78, 0.23, -0.45, ...]	Document retrieval, search

Notice how similar questions produce similar embeddings—this is what enables semantic matching and intelligent retrieval systems.

What are Sentence Embeddings Used For?



Question–Question Similarity

Identify when users ask the same question using different words. "How do I reset my password?" and "I forgot my login—help!" can be recognized as similar queries, preventing duplicate support tickets.



Query–Document Matching

Match user queries to relevant documents or knowledge base articles. When someone asks about "account security", the system retrieves documents about password policies, two-factor authentication, and security best practices.



Chat Memory & Context

Enable chatbots to recall relevant past conversation turns. If you mentioned "the problem with my billing" earlier, the bot can retrieve that context when you later say "Is it fixed yet?"



Duplicate Detection

Automatically flag duplicate content, repeated questions in forums, or similar bug reports. This keeps databases clean and helps teams avoid redundant work.

These applications form the backbone of modern semantic search engines, recommendation systems, and retrieval-augmented generation (RAG) architectures.

Vector Space & Semantic Similarity

Understanding Vector Space

Imagine a multi-dimensional space where every word and sentence occupies a specific position. This is **vector space**—a mathematical framework where embeddings exist as points or arrows.

In this space:

- **Position matters:** Similar meanings cluster together
- **Distance = difference:** Farther apart = less similar
- **Direction = concept:** Vector directions can represent relationships (e.g., king → queen parallels man → woman)

Key Concepts

Concept	Meaning
Embedding	A point in vector space
Similarity	How close two points are
Distance	Mathematical measure of difference
Dimension	Number of features (often 384–1536)

❏ Practical Example

In vector space, "doctor" and "nurse" sit close together because they're both medical professionals. "Doctor" and "engineer" are farther apart, but closer than "doctor" and "banana."

When you search for "medical professional", the system looks for all vectors within a certain distance of that query embedding—automatically finding doctors, nurses, surgeons, and other related concepts.

Cosine Similarity: Measuring Semantic Closeness

The Core Formula

Cosine similarity measures the angle between two vectors. Unlike Euclidean distance (straight-line distance), cosine similarity focuses on direction rather than magnitude. This makes it ideal for comparing text embeddings of different lengths.

$$\text{cosine similarity} = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}$$

The formula calculates the cosine of the angle between vectors **A** and **B**. Smaller angles mean higher similarity—the vectors point in nearly the same direction.

Why Cosine?

- Works well with high-dimensional data
- Normalized to range from -1 to 1
- Efficient to compute at scale
- Industry standard in NLP and search

Interpreting Similarity Scores

Score Range	Interpretation
0.9 – 1.0	Nearly identical meaning
0.7 – 0.89	Highly similar, related concepts
0.5 – 0.69	Moderately similar, some overlap
0.3 – 0.49	Weakly related
0.0 – 0.29	Different or unrelated
Below 0.0	Opposite meanings (rare)

In practice, a threshold of 0.7 or higher often indicates strong semantic similarity suitable for retrieval or matching tasks.

Practical Example: FAQ Matching in Action

Let's see how embeddings and cosine similarity solve a real-world problem: matching user questions to existing FAQ answers, even when the wording differs significantly.

The Scenario

A university help desk receives hundreds of questions daily. Many are variations of the same core inquiry. Traditional keyword matching fails because students phrase questions differently. Semantic similarity succeeds where exact matching fails.

Question Similarity Comparison

Question A	Question B	Similarity Score	Interpretation
"How do I reset my password?"	"I forgot my password, how can I recover it?"	0.89	Highly similar—both about password recovery
"Where is the library located?"	"How do I get to the library?"	0.82	Very similar—both seeking library directions
"When does registration open?"	"What are the library hours?"	0.31	Different topics—one about registration, one about hours
"How do I add a class?"	"What's the process for dropping a course?"	0.68	Moderately similar—both about course management

The system automatically routes similar questions to the same FAQ answer, preventing duplicate responses and improving support efficiency. A single comprehensive answer about password resets can satisfy dozens of differently-worded questions.

Limits, Ethics & Real-World Applications



Semantic Drift

Embeddings can conflate related but distinct concepts. "Bank" (financial institution) and "bank" (river edge) may cluster together, causing confusion. Context is critical but sometimes lost in compression.



Short Text Challenges

Very brief prompts lack sufficient context for accurate embedding. "Help" or "Info" are too vague —similarity matching fails without semantic detail. More descriptive queries yield better results.



Ambiguity Problems

Ambiguous phrasing reduces accuracy. "Can you fix this?" without specifying "this" produces generic embeddings that won't match relevant documents effectively.



Data Privacy & Ethics

Embeddings must be trained on appropriate, consented data. Using private conversations or copyrighted content without permission raises serious ethical concerns. Always ensure training data is public, licensed, or properly anonymized.

Key Limitations Summary

Limitation	Mitigation Strategy
Polysemy (multiple meanings)	Use context-aware models, larger text chunks
Short queries	Encourage users to add detail, expand queries automatically
Domain specificity	Fine-tune embeddings on domain-specific data
Computational cost	Cache embeddings, use efficient vector databases

Debrief, Lab Direction & Homework

Debrief Discussion

Let's solidify your understanding with three key questions:

1. **Why must text be converted into numbers?**

Machines can only process mathematical operations. Converting text to vectors enables computation, comparison, and pattern recognition.

2. **What's the difference between tokens and embeddings?**

Tokens are pieces of text (building blocks). Embeddings are meaningful numerical representations that capture semantic relationships.

3. **Share one real-life use of semantic similarity.**

Think about search engines, chatbots, recommendation systems, or duplicate detection in your own experience.

Homework Assignment

 Task: Campus FAQ Similarity Analysis

1. Select 3 frequently asked questions from your campus FAQ page
2. Identify which questions are semantically similar
3. Explain your reasoning using similarity logic and concepts from this module
4. Read the provided short note on RAG (Retrieval-Augmented Generation) basics

This exercise prepares you for building semantic search and retrieval systems in upcoming modules.

Thank You & What's Next

You've Mastered the Foundation

Thank you for completing Module 02! You've gained a crucial understanding of how AI transforms human language into the mathematical structures it can process.

You're now ready to control what AI does with that understanding.

 NEXT MODULE

Coming Up: Module 03 Prompt Engineering Fundamentals

- AI output quality depends on input quality
- Prompts guide what AI does and how it responds
- Learn principles of clear, safe prompt design
- Apply prompts to summarization, extraction, and classification

